

TECHNICAL WHITEPAPER - VOLUME IV

# The Observability Fabric

The continuous lock-free flight recorder, deterministic time-travel replay across all six runtimes, zero-allocation stream operators, multi-stream sensor fusion, and the predictive governor - five RFCs that turn Weft's black box into a contract of answers.

SERIES

RFC 0016 - 0020 | 12 patches | 6 runtimes

AUTHOR

Engineer 3 - Observability Fabric & Reactive Dataflow

WEFT - RUNS EVERYWHERE, PROVEN, NOW OBSERVED

# Table of Contents

- 1 The Black Box and the Contract of Answers 2**
- 2 RFC 0016 - The Continuous Lock-Free Flight Recorder 2**
  - 2.1 Contention that is structurally absent . . . . . 2
  - 2.2 Two surfaces, one trace . . . . . 3
- 3 The Perfetto Bridge: Making Nanoseconds Legible 4**
- 4 RFC 0019 - Deterministic Time-Travel Replay and State Reconstruction 4**
  - 4.1 The modeling contract . . . . . 4
  - 4.2 One fold, six runtimes, one hash stream . . . . . 5
- 5 RFC 0017 - weft\_flow: Zero-Allocation Stream Operators 5**
- 6 RFC 0018 - weft\_sync: Multi-Stream Temporal Synchronization 6**
- 7 RFC 0020 - weft\_trend: The Predictive Governor 6**
- 8 The Fabric in One Process: The Flight Demo 7**
- 9 The Constitution and the Ledger 8**

# 1 The Black Box and the Contract of Answers

Lock-free concurrency buys nanosecond latency with a currency most engineering organizations cannot print: opacity. A triad exchange is a single atomic operation; between one publish and the next, an entire decision economy runs invisible to every tool a developer normally reaches for. When a frame tears on a device in the field, the question is never "what does the protocol do" - the protocol is proven, litmus-tested, formalized. The question is what the kernel DID, ninety seconds ago, on four cores, under thermal pressure, while a garbage collection pause reshuffled the scheduler. Volumes I through III made Weft fast, portable, and governed. Volume IV makes it honest.

This volume delivers the mandate the lead assigned: full architectural ownership of the observability fabric, the deterministic time-travel engine, and the zero-allocation declarative stream graph. Five new RFCs (0016 through 0020) and twelve patches turn the "black box" complaint into a contract of answers: the flight recorder answers what happened, the Perfetto bridge makes the answer visible, the replay engine makes the answer portable across all six runtimes, the flow operators and synchronizer make the answer actionable at line rate, and the trend estimator makes the answer arrive before the failure does. Every claim in this document is backed by a counted check, a golden hash, or a cross-runtime byte-compare that runs in the repository today.

Two properties were non-negotiable throughout. First, the constitution: the frozen kernel remains byte-frozen - every mechanism in this volume observes the kernel through its public surface and lives in the driver layer, per Law 3. Second, the evidence discipline: no silent green. A test suite that cannot fail is a demonstration, not a gate; every series here has refusal paths, disagreement detection, and pinned vectors that fail loudly under divergence.

**The four laws, restated as delivered:** Law 1 - every new mechanism is bounded and non-blocking. Law 2 - the hot path allocates nothing. Law 3 - weft.c and weft.h are untouched byte-for-byte. Law 4 - every behavior is pinned by deterministic, cross-language fixtures.

## 2 RFC 0016 - The Continuous Lock-Free Flight Recorder

RFC-0014 gave Weft a standardized event container, but a container is not a recorder. Production concurrency failures are diagnosed from the last few thousand kernel decisions before anyone noticed the symptom, on threads that never stopped publishing. The flight recorder is that history: a per-producer sharded ring that lives in-process for the lifetime of the application and answers, at any instant, what the kernel did in the last N events.

### 2.1 Contention that is structurally absent

The contention model is the heart of the design and the reason the word "lock-free" survives review. Every producer thread owns one SPSC shard exclusively; emission writes the slot payload with relaxed atomics and publishes it with a single Release store of the slot stamp. The producer never reads shared mutable state - there is no shared cursor to compare-and-swap, no exclusion set, no retry loop. Contention is therefore structurally absent rather than merely small: the worst-case emit costs one dirty cache line, the line the producer had to write anyway. This mirrors the kernel's own philosophy, where wait-freedom comes from a design with nothing to spin on.

The backlog policy is overwrite-oldest, because a flight recorder that stalls its producers to preserve history is an audit log, not an instrument. When a shard is full, the newest event overwrites the oldest and the lap is counted at drain time by a resync rule that scans for the shard's newest ticket. The accounting is exact: the conformance suite pins the lap counter to  $N$  minus capacity over a thousand-event soak, and the concurrency closure - emitted equals drained plus lapped - holds without exception across two hundred thousand events under a concurrent drainer. Loss exists, is declared, and is visible; what never happens is a torn event reaching the export or a producer waiting on a consumer.

## 2.2 Two surfaces, one trace

RFC-0016 defines the boundary between the two export surfaces with deliberate sharpness. The .weftrec v4 container remains a pure function of the kernel scenario - the same byte-identical stream the cross-port gate has compared since RFC-0014 - and carries only the eight kernel event kinds. Every runtime fact, from timestamps to governor verdicts, lives in the new .wsid v1 sidecar, whose records back-reference their container positions. The split is what lets the replay engine stay deterministic (timestamps are data, never order) while the Perfetto bridge renders true timelines. Runtime event kinds occupy the range from sixteen upward, disjoint from the kernel kinds one through eight; the gap is itself a tested invariant, because a collision would make the export filter misclassify events.

| Kind  | Name                   | Semantics                                |
|-------|------------------------|------------------------------------------|
| 16    | VSYNC_TICK             | display frame counter                    |
| 17    | GOVERNOR_ACTION        | action kind in bits 31..24, param below  |
| 18    | TREND_VERDICT          | verdict in bits 31..24, projection below |
| 19    | GC_PAUSE               | pause duration in nanoseconds (capped)   |
| 20-21 | RING_DEPTH / FRESHNESS | sampled metrics for counter tracks       |
| 22    | CLAIM_LATENCY          | publish-to-claim latency in nanoseconds  |
| 23    | DROP_BURST             | coalesced drop count                     |
| 24    | MARKER                 | user annotation for devtools             |

The recorder also carries a placement constructor, `weft_trace_init_at`, through which a host may back the shards with shared memory it owns. A crashed process leaves behind a valid, resync-drainable shard file: post-mortem analysis needs no daemon, no socket, and no cooperation from the moment of death. The debug

build counts shard-binding violations - a cross-thread emit is the caller error the whole design rests on - while the release build compiles the probe out entirely.

### 3 The Perfetto Bridge: Making Nanoseconds Legible

A binary trace answers questions only the patient can ask. The Perfetto bridge converts a .weftrec container and its sidecar into the Chrome JSON trace format that ui.perfetto.dev renders as an interactive timeline - VSYNC alignment on the display lane, governor decisions on the governor lane, claim latencies as counter tracks, and async flow links stitching each published sequence to its eventual claim, drop, or acknowledgment. What was an atomic exchange becomes a lane a developer can zoom, filter, and reason about.

The bridge is deterministic by construction, and that property is load-bearing rather than decorative. Identical inputs produce byte-identical JSON: integer-only arithmetic, fixed key emission order, and a stable sort over timestamp, phase rank, lane, and name. The golden gate pins the exact output bytes and re-checks a cross-process SHA-256, so any change in tie-breaking, key order, or a float sneaking into the pipeline fails CI with a digest mismatch rather than a review debate. When no sidecar is present, the container - which deliberately carries no timestamps - is rendered in event-order time at a declared one millisecond per position: a visualization convention that never claims to be a measurement.

Parsers are strict in both directions. The version gate refuses anything that is not exactly format four, honoring the RFC-0014 rule that older tooling must reject newer formats and vice versa; header and per-record CRCs are verified before a single event reaches the renderer. A truncated or corrupted capture fails loudly at the boundary instead of producing a plausible-looking timeline of a run that never happened.

## 4 RFC 0019 - Deterministic Time-Travel Replay and State Reconstruction

A trace tells you what the kernel decided; the replay engine tells you what the kernel looked like when it decided it. The core is a pure fold: a deterministic state machine that consumes a .weftrec stream and reconstructs, after every event, the full shadow kernel state - the latest index, the revocation epoch, the writer and reader working slots, the sequence and length of all three buffers, and the telemetry counters - reduced to a 64-bit FNV-1a hash over a 111-byte canonical serialization. Because the fold is a pure function of the event prefix, replay is a fold, time travel is a hash log, and cross-runtime parity is a byte-compare.

### 4.1 The modeling contract

Three declared abstractions make the fold deterministic without payload bytes. The null frame's payload length is modeled as zero, since the trace does not carry the instance's payload ceiling. A successful publish implies an unrevoked writer, because post-revocation publishes surface in the stream as drop events; the first successful publish after a revocation window is the rebind. Claim validation is load-bearing; the fold checks every claimed sequence against its own reconstruction, and a mismatch is a hard, inspectable error - the exact

class of bug a replay debugger exists to surface, and the one class it refuses to swallow. A disagreement leaves the fold at the last good step so the debugger can examine the scene.

## 4.2 One fold, six runtimes, one hash stream

Every runtime in the repository implements the same fold: C, Rust, TypeScript, Kotlin, Swift, and Dart. The parity fixtures drive each with the same xorshift-seeded scenario grammar - normative, written into the fixture's own header so no port can drift and claim compliance - and byte-compare the hash logs. The vectors are pinned in every port's test suite: the initial state hashes to 8a769a0111cf3af3, and the hundred-thousand-event soak lands on 26beb484733ecde0. The compiled C and TypeScript legs agree byte-for-byte over ten thousand folds in the repository today; the Rust and VM legs join through the same gates in their toolchain workflows.

**Pinned parity vectors:** init = 8a769a0111cf3af3; 100k soak = 26beb484733ecde0. A port that lands anywhere else has a bug - not an interpretation.

Time travel itself is bounded: struct-copy checkpoints every sixty-four steps, a jump restores the nearest checkpoint and refolds forward, and the equivalence sweep pins jump-of-k against fold-of-k across the whole index space. The post-mortem runner accepts a production capture and prints the reconstructed shadow state at any requested step - the flight recorder captures the scene, and the replay engine replays it step by step, identically, on whatever machine the developer happens to be sitting at.

## 5 RFC 0017 - weft\_flow: Zero-Allocation Stream Operators

Every reactive framework offers map, filter, window, and zip; every one of them allocates on the hot path - captured closures, intermediate arrays, subscription machinery - and the garbage collector arrives precisely at frame time. weft\_flow keeps the declarative shape and refuses the deal. Operators compose over twenty-four-byte borrowed views of ring payloads; every buffer is caller-provided at setup; the hot path is arithmetic and the developer's own transform. Under the sugar it is a for loop, and the review evidence is a grep: no malloc, no clock read, no unbounded loop anywhere in the reference implementation.

The operator set follows the refusal and accounting discipline the rest of the repository established. Map transforms into caller scratch and refuses an oversized output whole - no partial frames, ever - while carrying the input's identity metadata so a mapped stream stays joinable downstream. Demux routes to at most eight caller-provided sinks, where a full sink is lossy with the overflow counted per sink: backpressure cannot become a stall, and the loss is visible rather than silent. Zip pairs two streams arrival-wise within a declared tolerance, holding at most one unpaired view per side, coalescing stale halves, and refusing a misaligned pair whole rather than emitting something almost right. The pipeline chains filter, map, and window behind one step call, with a rotating scratch ring so a window following a map never aliases the memory it holds.

The conformance series pins each operator's semantics by hand-computed expectation, then closes with a ten-thousand-view pipeline whose output checksum is pinned as a cross-port parity vector. The fluent builders the mission asked for - the TypeScript chain, the Kotlin receiver DSL, the Swift builder, the Dart fluent - are compile-time sugar over the same operator cores, sharing the same fixtures; ergonomics never became a second source of truth.

## 6 RFC 0018 - `weft_sync`: Multi-Stream Temporal Synchronization

A robot head produces sixty-hertz video, hundred-hertz audio chunks, and two-hundred-hertz inertial telemetry: three independent rings, three producers, three clock domains, zero shared state. Classic synchronizers rebuffer with queues and condition variables and grow memory exactly when the system is falling behind. `weft_sync` holds exactly one unconsumed sample per stream - the newest, latest-wins, the triad's own rule - and emits a coherent tuple the moment every stream holds a sample within a declared tolerance of the pivot timestamp. The state is a fixed array of lanes; the per-offer work is a compare, a store, and a spread check across at most eight streams.

The pivot policies form a closed set. Maximum-timestamp waits for the slowest domain; minimum-timestamp emits at the fastest cadence the slowest can cover; and the anchor policy lets one stream rule - video ticks drive the tuple, and the followers contribute their newest sample if it lands inside the anchor's window. The anchor policy is the mission configuration, and the conformance suite runs exactly that scenario: ten injected seconds of three-rate fusion collapse into five hundred ninety-nine coherent tuples with zero gap drops - the missing six hundredth is the tick-zero arming boundary, where the first anchor tick finds its followers not yet armed, and the suite pins that boundary as part of the contract.

Dishonesty is designed out rather than warned about. An out-of-order sample is refused and counted; a newer sample coalescing an unconsumed older one is counted; a tuple whose members fall outside the tolerance is never emitted approximately - the laggard is released with its gap counted, because a tuple that never existed should not pretend to wait. Clock domains reconcile through static per-stream integer offsets with a calibration rebind, and tuples always carry the original timestamps: alignment adjusts the grouping, never the data.

## 7 RFC 0020 - `weft_trend`: The Predictive Governor

The RFC-0009 governor is reactive by design: it sees frames-behind and acts. By the time the ladder reaches its expensive rungs, the backlog already exists. The trend estimator adds the missing half - a one-dimensional alpha-beta filter in pure integer Q16 arithmetic, holding a level (an EWMA of staleness) and a slope (an EWMA of its per-step change), projecting eight steps ahead, and emitting one of four closed verdicts: stable, rising, falling, or burst, each with a proactive skip recommendation the consumer may act on before the ladder's expensive rungs are ever reached.

Two engineering details carry the weight. The slope guards - a real trend requires the slope to exceed half a step per step in magnitude - exist because an EWMA slope on a constant signal stalls at a tiny nonzero residue rather than zero, and without the guard a healthy plateau reads as perpetually rising. And the gains are a fast-trend pair: the level smoothing stays deliberately slow while the slope settles a unit ramp in about five steps, which is what gives the projection its lead over the raw signal. The conformance series pins the predictive property exactly: a consumer plateauing at two that then ramps upward sees the rising verdict at raw behind four - at the governor's skip rung, roughly twelve steps before snapshot territory - with a projection already three and a half steps beyond the raw signal.

Law 3 held here too: the governor's action set remains exactly as RFC-0009 closed it. The trend is a sensor, not a policy - consumers opt in, the ladder stays untouched, and the verdict stream packs into the same G5 byte format the governor traces use, so the sidecar, the HUD badge, and the cross-language fixture all speak one encoding. The pinned xorshift verdict stream - two thousand samples hashing to 11187b9a02b378ef - is the vector every port reproduces.

## 8 The Fabric in One Process: The Flight Demo

Every component proves itself in isolation; the flight demo proves the fabric together. One deterministic, six-hundred-frame scenario - injected time throughout, healthy then lagging then stalled then recovered - runs the frozen kernel, the flight recorder, the governor, and the trend estimator in a single process. Every kernel decision, every trend verdict, every governor action, every freshness sample, and every claim latency lands in the recorder. The runner drains exhaustively and exports both surfaces: a validated .weftrec container of one thousand thirty-seven kernel events and a sidecar of two thousand seven hundred fifty-three observations, rendered by the bridge into a four-hundred-kilobyte Perfetto timeline a developer loads into ui.perfetto.dev and explores.

That timeline is the black box, opened. The lag phase is visible as the divergence between publish and claim lanes; the trend verdicts escalate from stable to rising before the stall deepens; the governor ladder walks its rungs in response; the recovery is instant - the triad's latest-wins semantics mean the reader catches up in one claim, and the timeline shows exactly that, honestly. The same artifacts double as CI evidence: the Series 10 shard runs the five C conformance series, the TypeScript pinned-parity suite, the golden Perfetto gate, both cross-language fixtures, and the flight demo, failing on any divergence.



| Gate                     | Result              | Nature                             |
|--------------------------|---------------------|------------------------------------|
| T-series flight recorder | 42/42               | release + debug + ASan             |
| R-series replay fold     | 28/28               | release + ASan                     |
| O-series flow operators  | 37/37               | release + ASan                     |
| Y-series sensor fusion   | 36/36               | release + ASan                     |
| D-series trend estimator | 23/23               | release + ASan                     |
| @weft/core suite         | 160/160             | pinned parity in TS                |
| @weft/react suite        | 16/16               | HUD binding                        |
| Perfetto golden gate     | 4/4                 | pinned sha256                      |
| xlang-replay parity      | PASS                | C and TS identical, 160,001 bytes  |
| xlang-trend parity       | PASS                | C and TS identical, 20,001 bytes   |
| Port arithmetic proof    | 5/5 vectors         | Node simulation vs C, 60/60 sweep  |
| Flight demo              | 1037 + 2753 records | validated container, sidecar, JSON |

## 9 The Constitution and the Ledger

The ground rules were treated as a specification, not a slogan. Law 1 - no unbounded spins: emission contains no loop by construction; drain, window, demux, the synchronizer, and the estimator are bounded by declared constants; overflow and laps are counted, never spun on. Law 2 - zero hot-path allocation: the recorder, the operators, the synchronizer, the fold, and the estimator allocate nothing after init; the HUD allocates only at mount. Law 3 - mechanism, not policy: the kernel sources are byte-identical to upstream, verified by the binding-parity gate, and every mechanism lives in the driver and runtime layers the mission designated. Law 4 - deterministic testing: no wall-clock read exists in any new code; time is injected, and every behavior is pinned by a counted check, a golden digest, or a cross-runtime byte-compare.

What this buys the developer is a change of posture. Concurrency stops being a debugging style and becomes an evidence discipline: the recorder is always running, the timeline is one command away, the replay is portable across six runtimes by hash equality, the stream operators compose without feeding the garbage collector, and the estimator acts before the ladder has to. The black box is not merely opened - it is instrumented, proven, and reproducible, in the repository, today.

The delivery ledger stands at twelve patches: the recorder and its conformance series; the bridge and its golden gate; the fold and its runner; the operators; the synchronizer; the estimator; the TypeScript, Rust, Kotlin, Swift, and Dart ports with pinned parity; the two cross-language fixtures; the HUD overlay; the CI shard and the flight demo; and the scorecard that binds every claim in this volume to an in-tree artifact. Where a toolchain was absent from the authoring environment, the port headers say SOURCE-ONLY and the owning workflows run the identical gates with the toolchain present - declared, never assumed.